

Windows应用程序与MFC基础

党宇

dangyu@nankai.edu.cn



机器人与信息自动化研究所

Institute of Robotics & Automatic Information System



南开大学

Nankai University

主要内容

- 初识可视化编程
- Windows应用程序
 1. Windows程序开发流程
 2. 事件与消息
 3. Windows应用程序的基本结构
 4. 应用程序举例
- MFC基础知识
 1. MFC类的组织结构
 2. MFC框架程序分析
 3. MFC消息映射机制

主要内容

- 初识可视化编程
- Windows应用程序
 - 对应课本：第三章
 - 补充内容：《深入浅出MFC》第一章
- MFC基础知识
 - 对于课本：第八章
 - 补充内容：《VC++深入详解》第三、四章



初识可视化——基本概念

➤ 如何开发可视化程序？

- 使用Windows系统提供的Windows API
- 使用微软提供的MFC

➤ Windows API

- API: Application Programming Interface (应用程序编程的接口)，每个 Windows 应用程序都直接或间接地使用了 Windows API
- SDK: Software Development Kit (软件开发包)，通常包含API 函数库、帮助文档、使用手册、辅助工具等资源
- MSDN: Microsoft Developer Network, 是面向软件开发者的一种信息服务



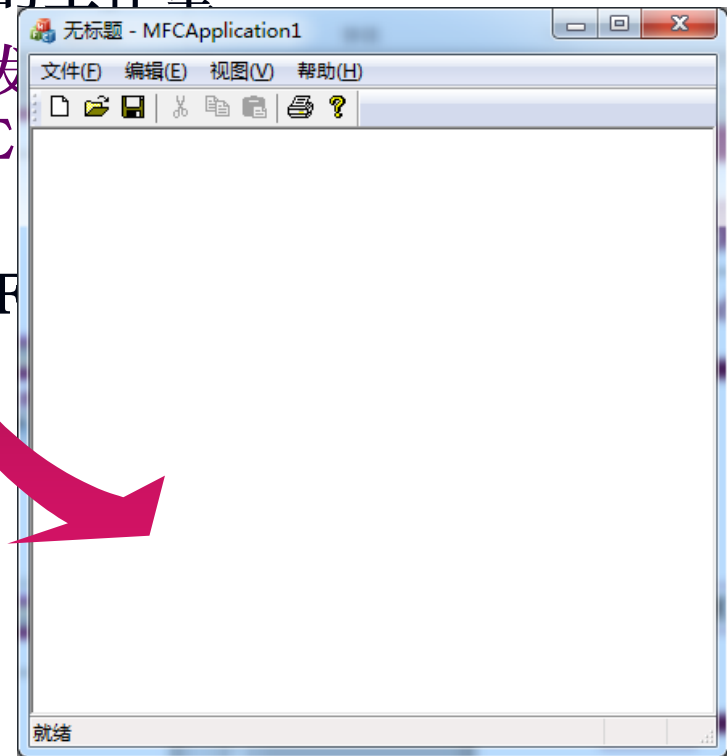
初识可视化——基本概念



classes (微软基础类库),
API, 并且包含一个应用程
的工作量

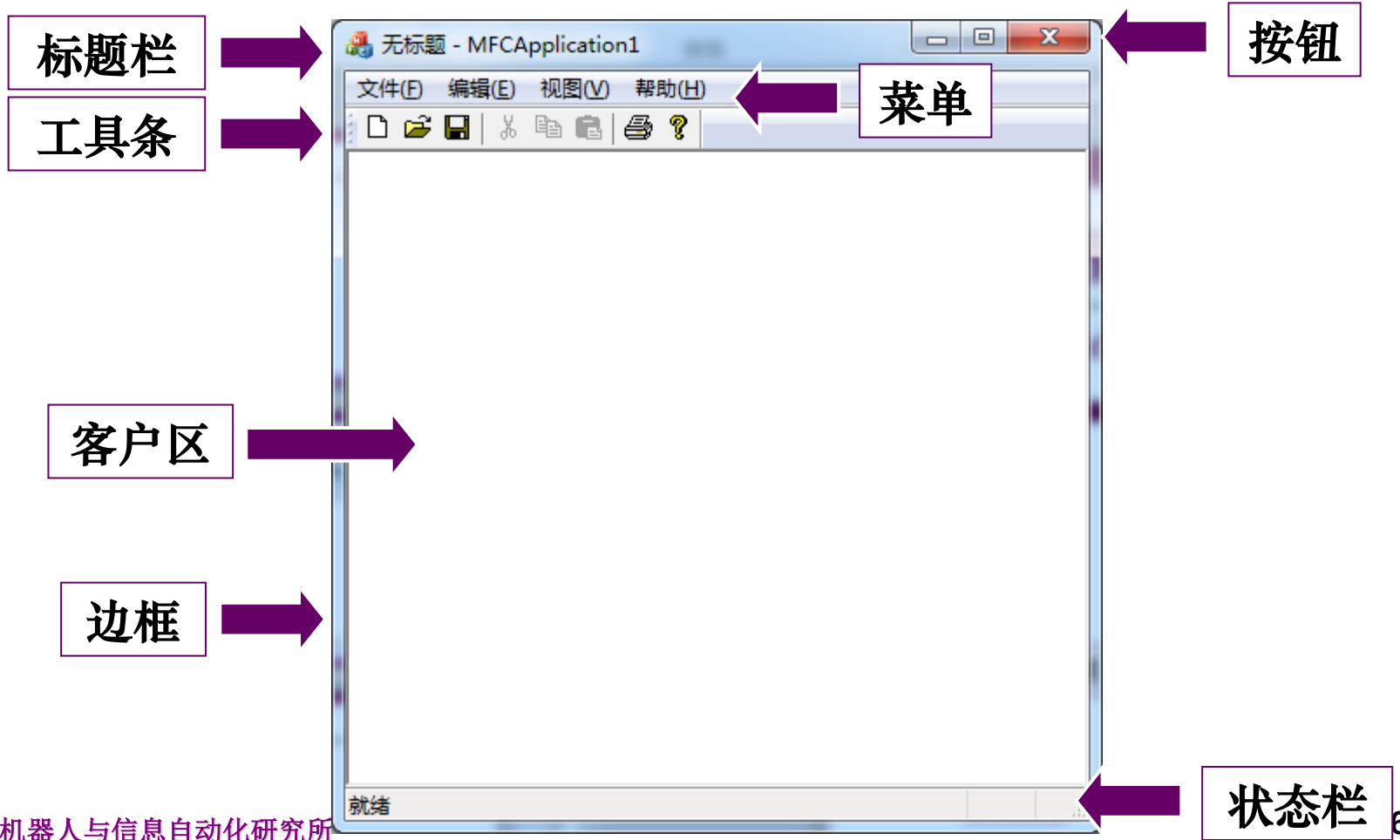
发
E

IF



初识可视化——窗口

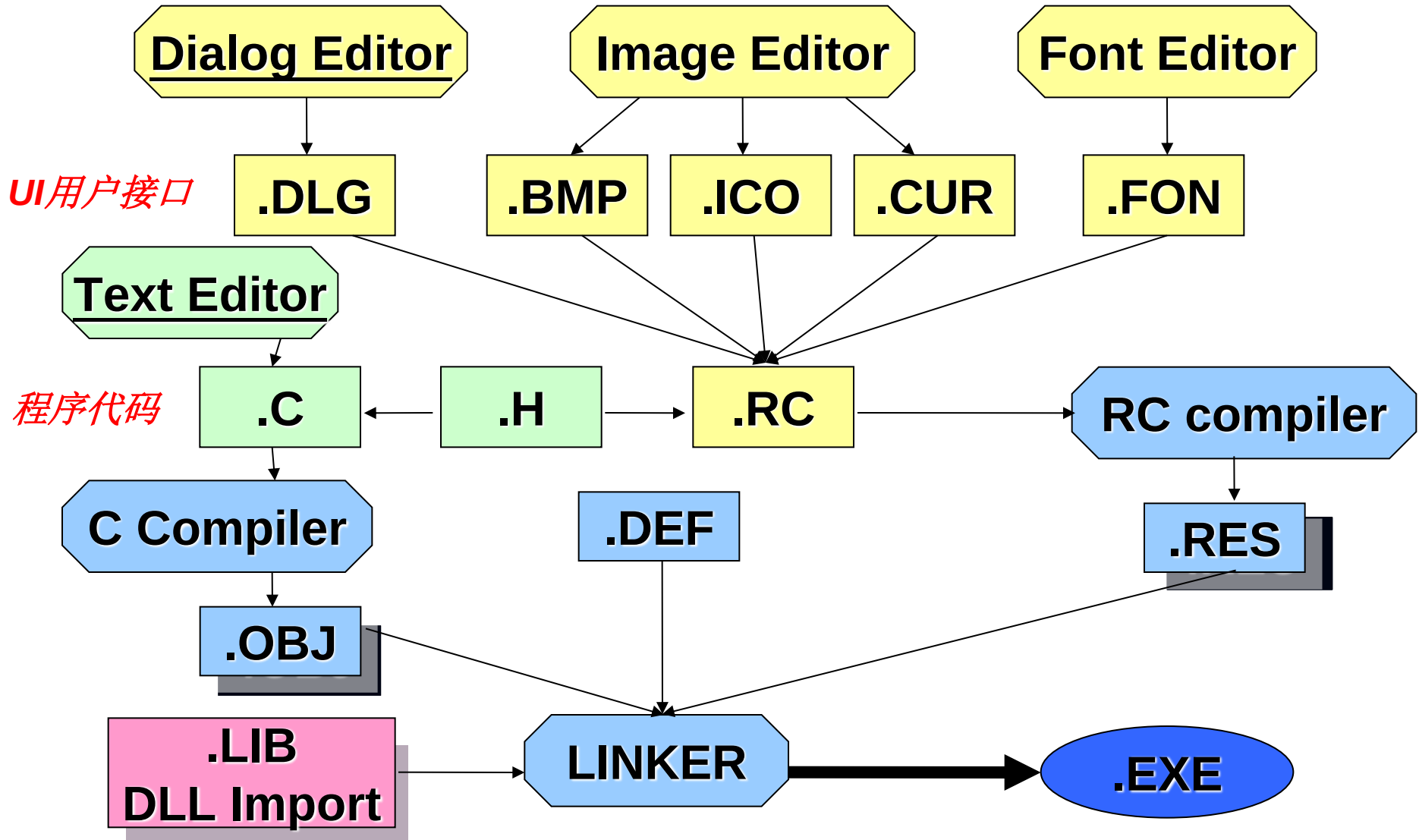
- 窗口(Window)是程序用户界面，负责应用程序与用户交互



主要内容

- 初识可视化编程
- Windows应用程序
 1. Windows程序开发流程
 2. 事件与消息
 3. Windows应用程序的基本结构
 4. 应用程序举例
- MFC基础知识

1、Windows程序开发流程

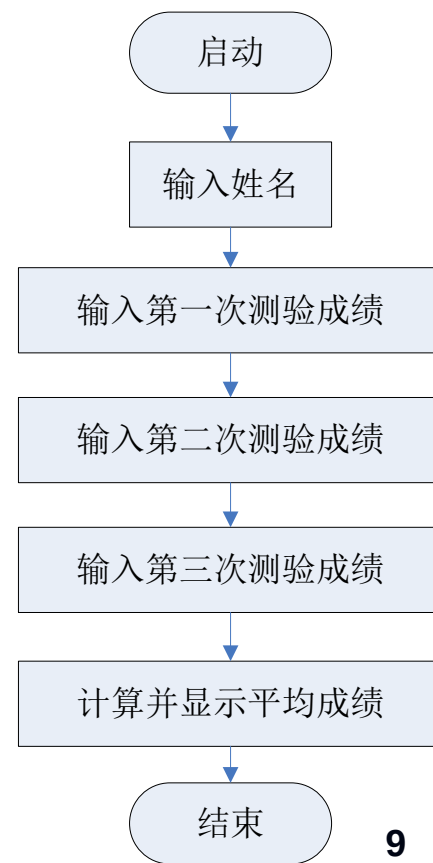


2、事件与消息

以消息为基础，以事件驱动之
(message based, event driven)

➤ 2.1 事件驱动

- 举例：计算学生平均成绩
- DOS的**过程驱动**
 - 程序有明显的开始、明显的过程和明显的结束
 - 在等待用户输入过程中，所有输入设备(资源)都被“等待”过程抢占了

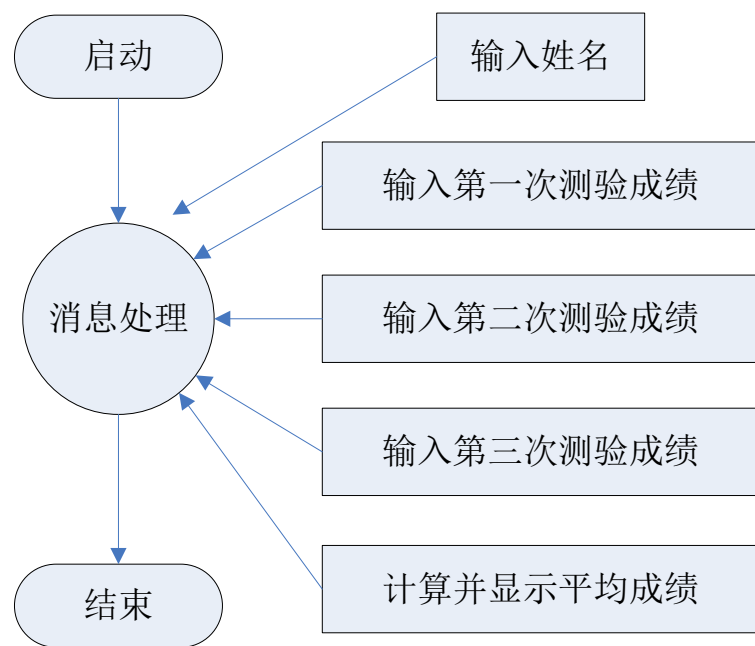


2、事件与消息

以消息为基础，以事件驱动之
(message based, event driven)

➤ 2.1 事件驱动

- 举例：计算学生平均成绩
- Windows的**事件驱动**
 - 围绕消息的产生与处理展开
 - 用户不断进行各种输入，计算机不断响应，形成“输入—响应”的程序结构



2.1 事件驱动

以消息为基础，以事件驱动之
(message based, event driven)

➤ Windows程序的消息响应机制

➤ 消息从何而来？

Windows操作环境中，无论是系统产生的动作或用户运行应用程序产生的动作，称为事件(Events)产生的消息(Message)

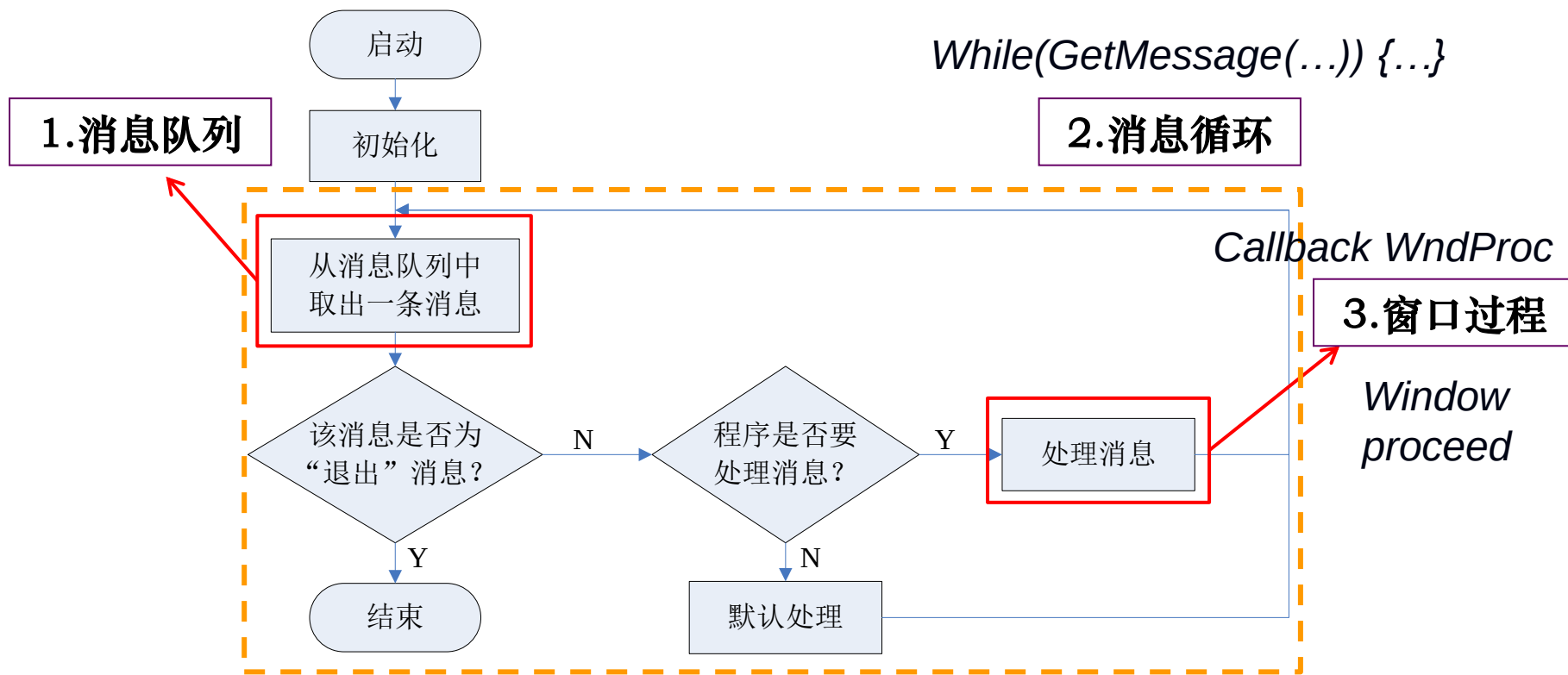
➤ 事件又从何而来？(请给出一个例子)

- 通过输入设备，如键盘和鼠标
- 通过屏幕上可视的对象，如菜单、工具栏按钮、滚动条和对话框上的控件
- 来自Windows内部

2.1 事件驱动

以消息为基础，以事件驱动之 (message based, event driven)

➤ 基于事件驱动的程序流程

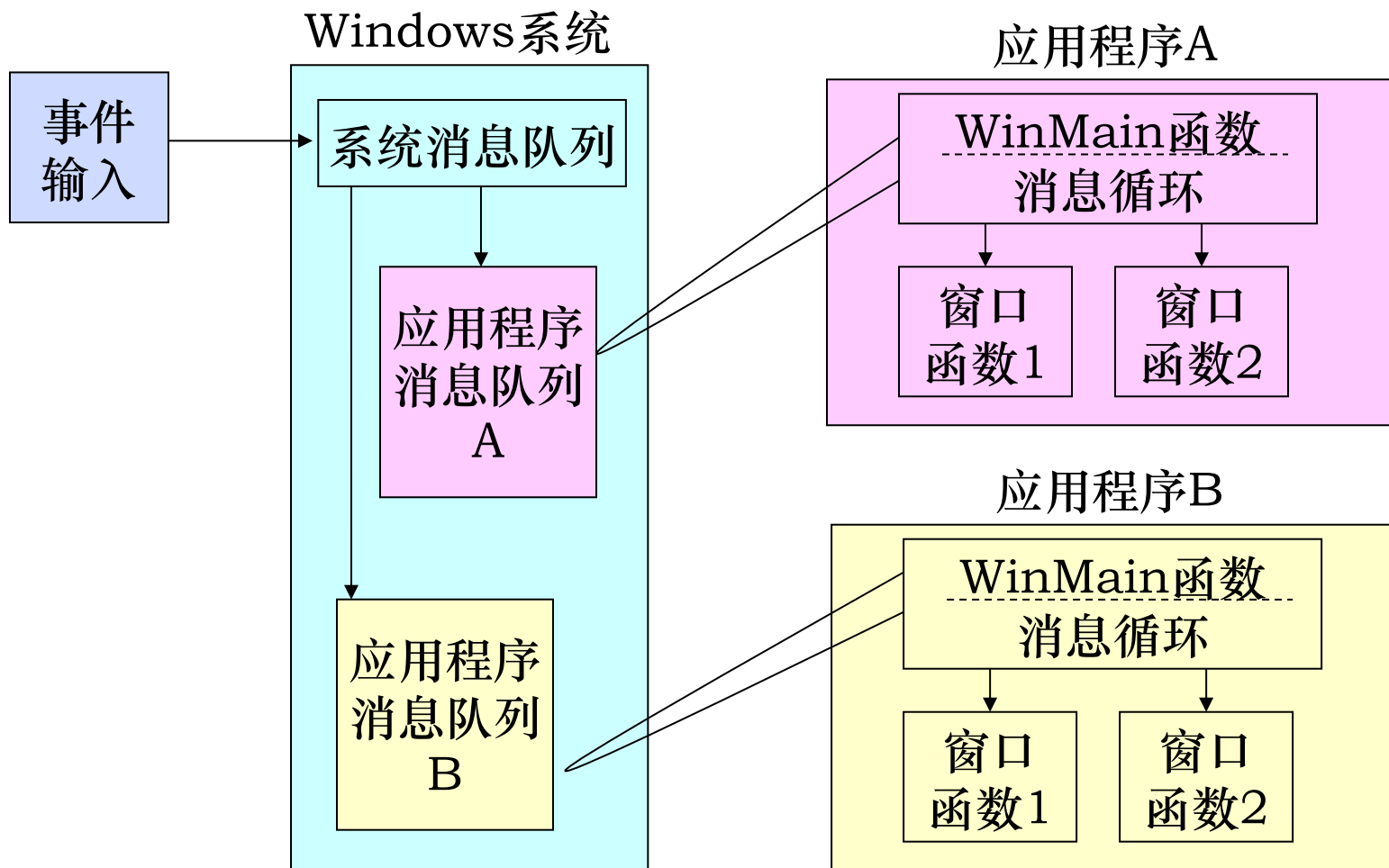


2.2 消息处理机制

- **Window消息系统的组成：**
 - **消息队列：**是系统定义的内存块，Windows有一个系统消息队列，并为每一个应用程序维护一个消息队列
 - **消息循环：**应用程序通过消息循环机制，从消息队列中获取消息，把它分派给适当的窗口，然后继续从消息队列中获取下一条消息，分派给适当的窗口，依次进行
 - **窗口过程：**利用窗口函数接收消息并响应消息

2.2 消息处理机制

- Window消息系统的组成：消息队列、消息循环、窗口过程



1) Windows消息

消息的数据结构

窗口句柄

系统程序

```
typedef struct tagMSG{  
    HWND hwnd;           //用以检索消息的窗口句柄  
    UINT message;        //消息值  
    WPARAM wParam;       //包含有关消息的附加信息  
    LPARAM lParam;       //随不同的消息而有所不同  
    DWORD time;          //消息送到队列的时间  
    POINT pt;            //消息发送时，屏幕光标的位置  
} MSG;
```

句柄 (Handle) : Windows标识不同对象或实例而使用的32位整数，应用程序通过句柄可访问相应的对象信息，所有的句柄类型以H开头

1) Windows消息

➤ 常用的Windows消息

WM_LBUTTONDOWN: 单击鼠标左键时产生的消息

WM_LBUTTONUP: 放开鼠标左键时产生的消息

WM_RBUTTONDOWN: 单击鼠标右键时产生的消息

WM_LBUTTONDBLCLK: 双击鼠标左键时产生的消息

WM_RBUTTONDBLCLK: 双击鼠标右键时产生的消息

WM_CREATE: 创建窗口时, `CreateWindow()`产生的消息

WM_CLOSE: 关闭窗口时产生的消息

WM_DESTROY: 关闭窗口时, `DestroyWindow()`产生的消息

WM_QUIT: 退出应用程序时, `PostQuitMessage()`发出的消息

WM_PAINT: Windows系统需要重绘时产生的消息

1) Windows消息

➤ 系统定义的消息分类

消息前缀	说明	消息前缀	说明
BM	按钮控件	LB	列表框控件
CB	组合框控件	SBM	滚动条控件
EM	编辑控件	WM	窗口

2) 消息循环

➤ 消息循环的格式

系统程序

```
MSG msg;
...
while(GetMessage(&msg, NULL, 0, 0))    //从消息队列中读取
{                                       //消息，存入Msg
    TranslateMessage(&msg);           //将虚拟键转化为字符信息
    DispatchMessage(&msg);           //将消息传送到指定窗口函数
}
```

- 当函数GetMessage()返回值为0，即检索到WM_QUIT消息时，程序将结束循环并退出

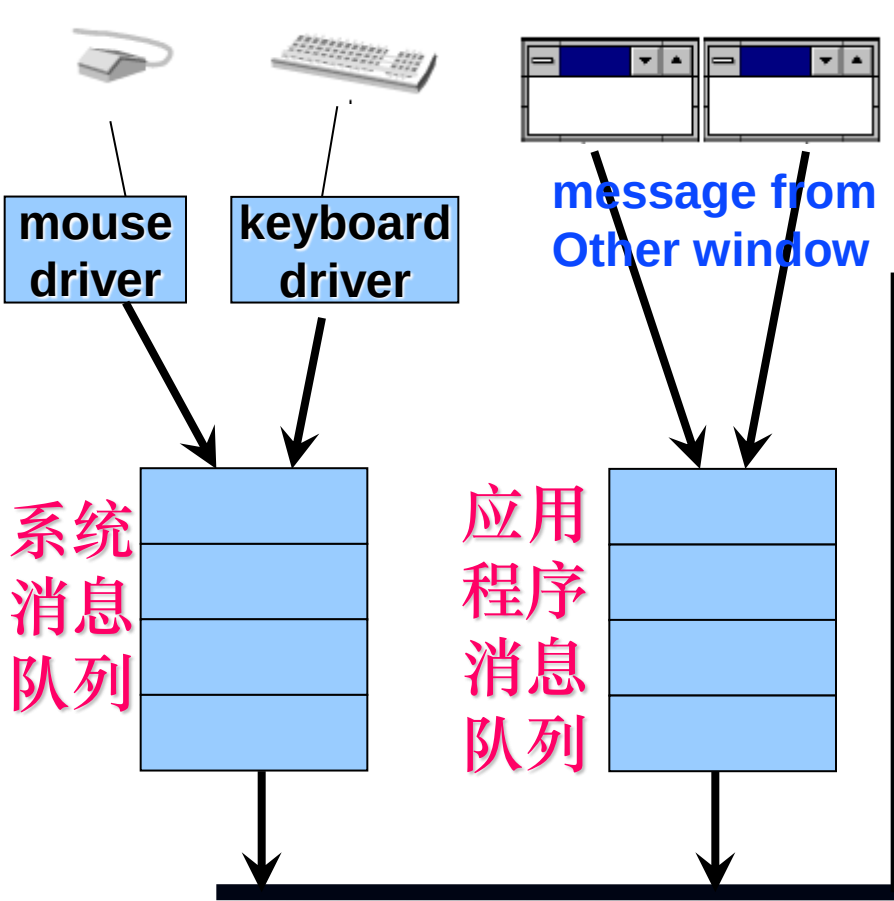
3) 窗口函数

回调函数

- 窗口函数是应用程序处理接收到的消息的函数
- 窗口函数的形式

```
LRESULT CALLBACK WndProc(HWND hWnd, UINT message, WPARAM  
    wParam, LPARAM lParam) {  
    ...  
    switch(message) {  
        case ...  
            break;  
        ...  
        case WM_DESTROY: PostQuitMessage(0); break;  
        default: return DefWindowProc(hWnd, message, wParam, lParam)  
    }  
    return 0;  
}
```

系统程序



WinMain ()

```
{  
    初始化;  
    while(GetMessage(&msg)){  
        TranslateMessage( );  
        DispatchMessage( );  
    }  
}  
    system  
    WndProc(msg)  
    {  
        switch(msg){  
        case MSG1:  
        case MSG2:  
        }  
        return 0;  
    }
```

概括：Windows应用程序的基本结构



➤ WinMain()——程序入口点

- 定义并注册窗口类：RegisterClassEx()
- 创建并显示窗口：CreateWindow()/ShowWindow()
- 消息循环：
GetMessage()/TranslateMessage()/DispatchMessage()

➤ WndProc()——窗口函数（回调函数）

- 使用switch-case区分消息
- 根据消息触发相应处理
- 调用DefWindowProc()处理应用程序不处理的消息及需要系统进一步处理的消息

3、Windows应用程序的基本结构

- 程序入口函数WinMain()和窗口函数WndProc()
- WinMain()的功能
 - 定义并注册窗口类、建立窗口并初始化
 - 进入消息循环，根据消息队列接收的消息，调用相应的处理过程
 - 接收WM_QUIT消息时，终止程序运行

系统程序

```
int APIENTRY _tWinMain (  
    HINSTANCE hInstance,           //应用程序当前实例句柄  
    HINSTANCE hPrevInstance,       //应用程序其他实例句柄  
    LPSTR lpCmdLine,               //命令行参数指针  
    int nCmdShow)                  //窗口显示方式
```



窗口的初始化

1. 定义窗口类：给窗口类数据结构WNDCLASSEX赋值

窗口的初始化

系统程序

```
typedef struct tagWNDCLASSEX {  
    UINT cbSize;                // 窗口类的结构大小  
    UINT style;                 // 窗口类的样式  
    WNDPROC lpfnWndProc;        // 窗口函数指针  
    int cbClsExtra;             // 类附加内存  
    int cbWndExtra;             // 窗口附加内存  
    HINSTANCE hInstance;        // 程序实例句柄  
    HICON hIcon;                // 图标句柄  
    HCURSOR hCursor;            // 光标句柄  
    HBRUSH hbrBackground;       // 背景刷句柄  
    LPCTSTR lpszMenuName;       // 菜单资源名  
    LPCTSTR lpszClassName;      // 窗口类名  
    ...  
} WNDCLASSEXW;
```


窗口的初始化

1. 定义窗口类：给窗口类数据结构WNDCLASSEX赋值
2. 注册窗口类：由函数RegisterClassEx()实现

系统程序

```
ATOM MyRegisterClass(HINSTANCE hInstance)
{
    WNDCLASSEX wcex;
    wcex.cbSize = sizeof(WNDCLASSEX);
    wcex.style          = ...;
    wcex.lpfnWndProc    = WndProc;    // 指定窗口函数
    ...
    wcex.hIcon          = LoadIcon (...);
    wcex.hCursor        = LoadCursor (...);
    ...
    return RegisterClassEx(&wcex);
}
```

窗口的初始化

- 3. 创建窗口：通过CreateWindow()创建窗口类的实例
- 4. 显示窗口：由ShowWindow()和UpdateWindow()实现

```
BOOL InitInstance(HINSTANCE hInstance, int nCmdShow) {  
    HWND hWnd;  
    hInst = hInstance; // 将实例句柄存储在全局变量中  
    hWnd = CreateWindow(szWindowClass, szTitle,  
        WS_OVERLAPPEDWINDOW, CW_USEDEFAULT, 0,  
        CW_USEDEFAULT, 0, NULL, NULL, hInstance, NULL);  
    ...  
    ShowWindow(hWnd, nCmdShow);  
    UpdateWindow(hWnd); //更新并绘制用户区，并发出WM_PAINT消息  
    ...  
}
```

系统程序

窗口的初始化

➤ CreateWindow()函数说明

系统程序

```
HWND CreateWindow (  
    LPCTSTR lpszClassName,    // 窗口类名  
    LPCTSTR lpszTitle,        // 窗口标题  
    DWORD dwStyle,            // 窗口样式  
    int X, int Y,              // 窗口左上角坐标  
    int nWidth, int nHeight,   // 窗口宽度高度  
    HWND hwndParent,          // 父窗口句柄  
    HMENU hMenu,              // 主菜单句柄  
    HINSTANCE hInstance,      // 当前程序实例  
    LPVOID lpParam             // 传递参数指针  
)
```

总结：Windows应用程序的基本结构



➤ WinMain()——程序入口点

- 定义并注册窗口类：RegisterClassEx()
- 创建并显示窗口：CreateWindow()/ShowWindow()
- 消息循环：
GetMessage()/TranslateMessage()/DispatchMessage()

➤ WndProc()——窗口函数（回调函数）

- 使用switch-case区分消息
- 根据消息触发相应处理
- 调用DefWindowProc()处理应用程序不处理的消息及需要系统进一步处理的消息

总结：Windows应用程序的生与死



以消息为基础，以事件驱动之
(message based, event driven)

➤ 消息响应机制

- 打开电脑，运行visual studio
- 新建 win32项目（或windows桌面应用程序）

消息队列



```
WinMain() {  
    MSG msg;  
    MyRegisterClass(...);  
    InitInstance (...) → CreateWindow()  
    while (GetMessage(&msg, ...)) {  
        TranslateMessage(&msg);  
        DispatchMessage(&msg);  
    }  
    return (int) msg.wParam; }
```

WM_CREATE

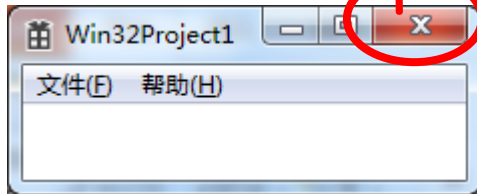
```
WndProc(hwnd, msg, wParam, lParam){  
    switch (msg) {  
        case WM_CREATE: ...  
        case WM_LBUTTONDOWN: ...  
        case WM_PAINT: ...  
        case WM_DESTROY: ...  
            PostQuitMessage(0);  
            break;  
        default: return DefWindowProc(...);  
    }  
    return 0; }
```

WM_QUIT

WM_CLOSE

WM_DESTROY

```
case WM_CLOSE:  
    DestroyWindow(...);
```



②

4

⑧

⑤

⑦

⑥

①

③



总结：Windows应用程序的生与死

1. 程序初始化过程中调用`CreateWindow()`创建窗口，`CreateWindow()`送出`WM_CREATE`消息给窗口函数，后者可以在此时做些初始化操作；
2. 在程序运行过程中，`GetMessage()`不断从消息队列中取得消息。如果消息是`WM_QUIT`，`GetMessage()`返回值为0，结束循环，进而结束整个程序；
3. `DispatchMessage()`把消息分派至窗口函数，消息将被处理；
4. 程序不断循环2和3；
5. 当使用者关闭窗口时，系统送出`WM_CLOSE`，通常窗口程序不处理它，由`DefWindowProc()`处理；
6. `DefWindowProc()`处理`WM_CLOSE`，调用`DestroyWindow()`把窗口清除，送出`WM_DESTROY`；
7. 窗口程序对`WM_DESTROY`的处理是调用`PostQuitMessage()`；
8. `PostQuitMessage()`送出`WM_QUIT`，准备让消息循环中的`GetMessage()`取得，如步骤2，结束消息循环。

Windows数据类型

- Windows将使用的数据类型定义在windows.h文件中

Windows数据类型	对应的基本数据类型(C语言)	说明
LONG	long	32位有符号整数
DWORD	unsigned long	32位无符号整数，段地址和相关的偏移地址
UINT	unsigned int	32位无符号整数
BYTE	unsigned char	8位无符号整数
BOOL	bool	布尔值
LPCSTR	const char *	指向字符串常量的32位指针
LPSTR	char *	指向字符串的32位指针
LPVOID	void *	指向未定义类型的32位指针

- 若数据类型前缀是P或LP，表示该类型是指针或长指针数据类型
- 若数据类型前缀是U，表示该类型是无符号数据类型

变量命名约定



表 1.1 匈牙利表示法

前 缀	含 义
a	数组
b	布尔值 (int)
by	无符号字符 (字节)
c	字符 (字节)
cb	字节记数
rgb	保存 RGB 颜色值的长整型
ex,cy	短整型 (计算 x,y 的长度)
dw	无符号长整型
fn	函数
h	句柄
i	整数 (integer)
m_	类的数据成员
n	短整型或整型
np	近指针
p	指针



变量命名约定



续表

前 缀	含 义
l	长整型
lp	长指针
s	字符串
sz	以零结束的字符串
tm	正文大小
w	无符号整型
x,y	无符号整型（表示 x 或 y 的坐标）

Windows数据类型

➤ Unicode编码

- 在头文件tchar.h中通过统一形式，兼容处理传统字符编码与Unicode编码

编码类型	常量表达形式	变量表达形式
SBCS/MBCS	“string”	char
Unicode	L “string”	wchar_t
统一处理	_T(“string”)	TCHAR

数据结构

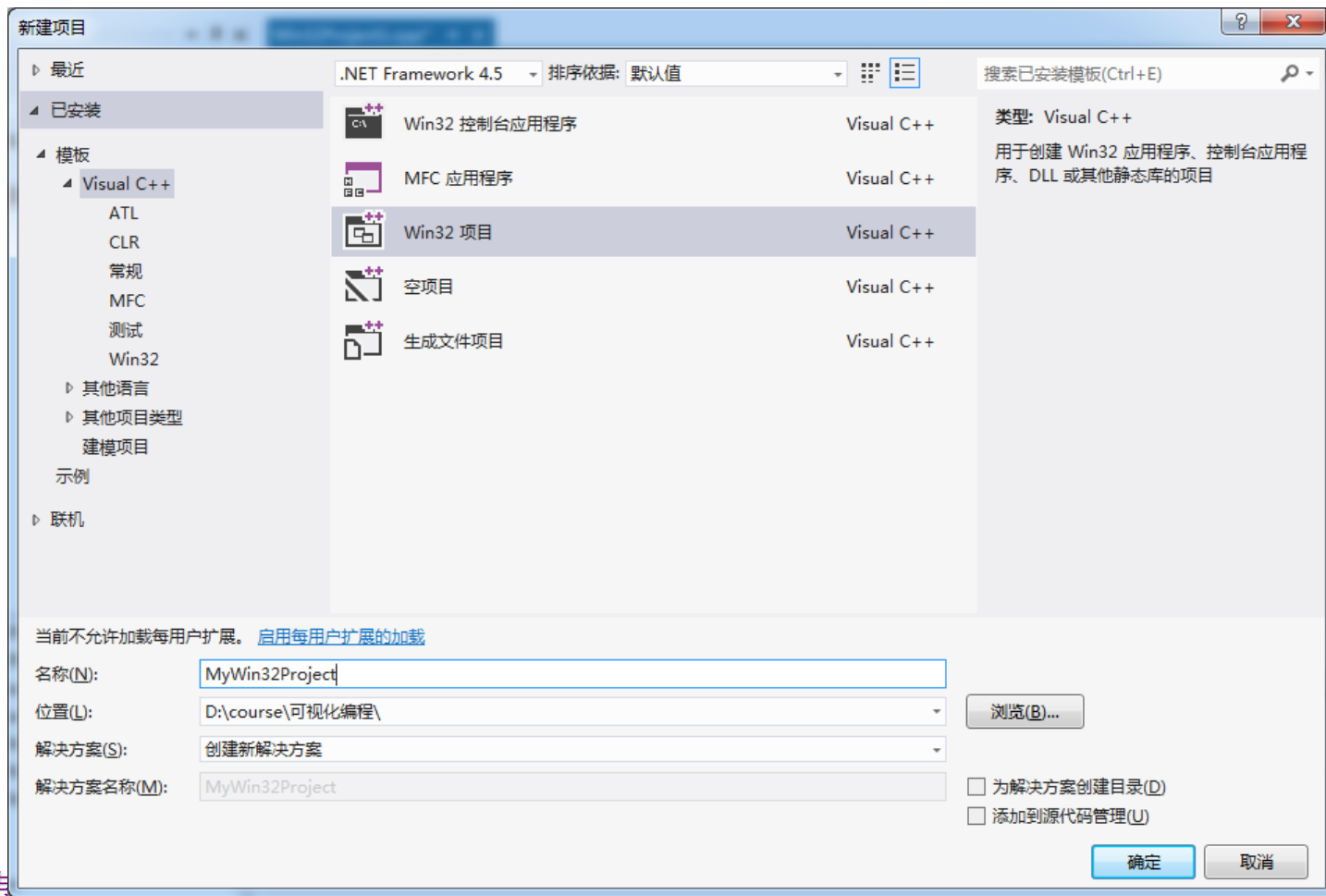
- 消息数据结构 MSG
- 窗口类数据结构 WNDCLASSEXW
- 点坐标结构 POINT
- 矩形区域结构 RECT

```
typedef struct tagPOINT
{
    LONG x; // 屏幕横坐标
    LONG y; // 屏幕纵坐标
} POINT;
```

```
typedef struct _RECT
{
    LONG left; // 矩形框左上角x坐标
    LONG top; // 矩形框左上角y坐标
    LONG right; // 矩形框右上角x坐标
    LONG bottom; // 矩形框右上角y坐标
} RECT;
```

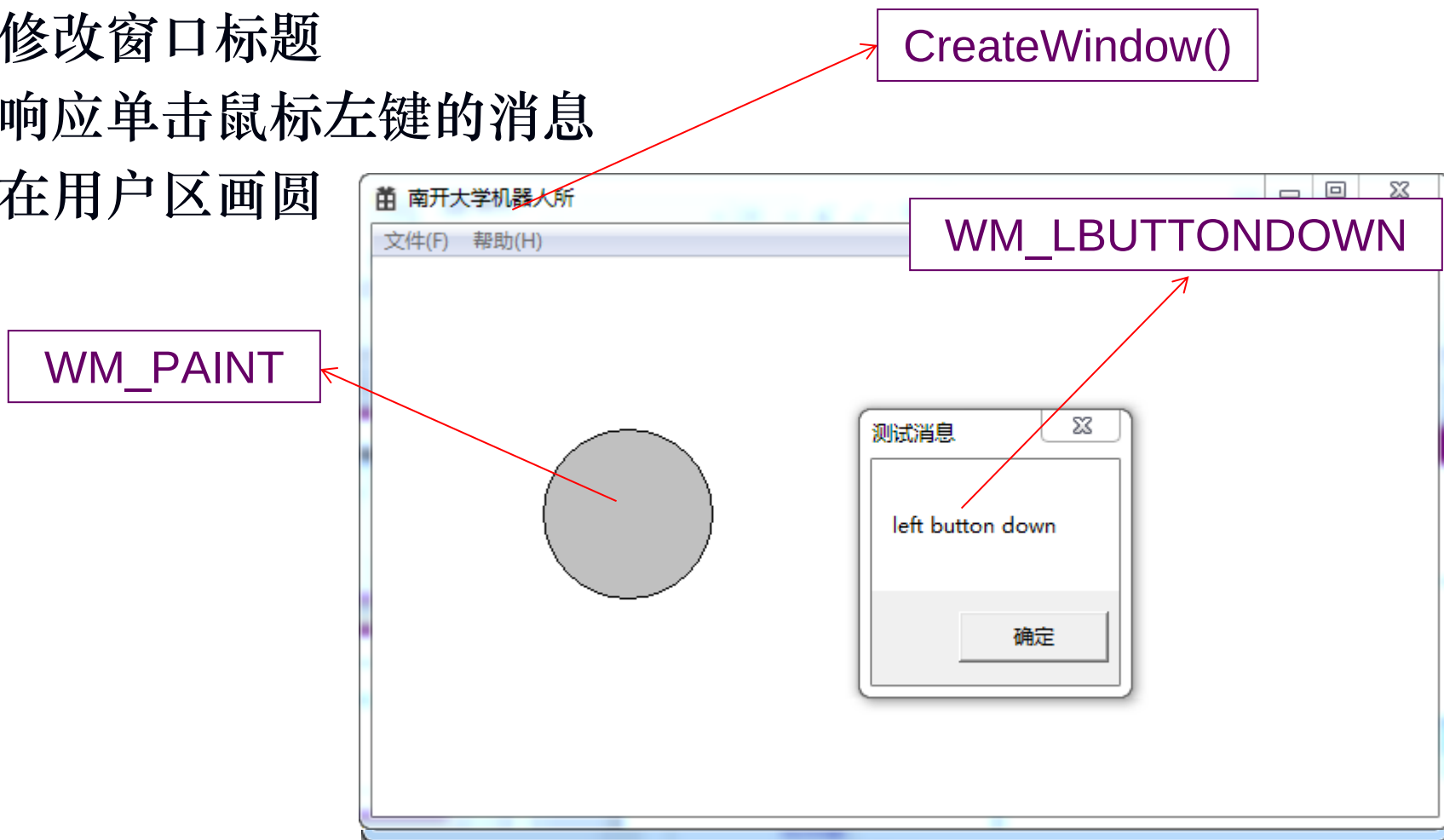
4、应用程序举例

➤ 通过Visual Studio 生成windows桌面应用程序



4、应用程序举例

- 修改窗口标题
- 响应单击鼠标左键的消息
- 在用户区画圆



4、应用程序举例

➤ 修改窗口标题

InitInstance()函数内

```
hWnd = CreateWindow(szWindowClass, _T("南开大学机器人所"),  
    WS_OVERLAPPEDWINDOW, CW_USEDEFAULT, 0,  
    CW_USEDEFAULT, 0, NULL, NULL, hInstance, NULL);
```

➤ 响应单击鼠标左键的消息

WndProc()函数内

```
case WM_LBUTTONDOWN:  
    MessageBox(NULL, _T("left button down"), _T("测试消息"), NULL);  
    break;
```

4、应用程序举例

➤ 在用户区画圆

WndProc()函数内

```
HBRUSH hBrush,oldBrush;           // 画刷
...
case WM_PAINT:
    hdc = BeginPaint(hWnd, &ps);    // 获取设备环境
    // 获取淡灰色画刷，并选择画刷（一般需要保留原始对象）
    hBrush = (HBRUSH)GetStockObject(LTGRAY_BRUSH);
    oldBrush = (HBRUSH)SelectObject(hdc,hBrush);
    // 绘制圆
    Ellipse(hdc, 100, 100, 200, 200);
    SelectObject(hdc,oldBrush);      // 恢复原始图形对象
```


主要内容

- 初识可视化编程
- Windows应用程序
 1. Windows程序开发流程
 2. 事件与消息
 3. Windows应用程序的基本结构
 4. 应用程序举例
- MFC基础知识